My Module: Admin Dashboard (End-to-End) — 2-Day Build Plan

Scope: Full stack — DB schema for admin-owned entities, backend APIs, and frontend UI for the Admin role only. This is what you present to your team as your task ownership.

---

0. Requirements & Specifications (share this with your team first)

Functional Requirements

ID	Requirement
FR-1	Admin can log in and see a dashboard restricted to admin-only routes
FR-2	Admin can create a new scholarship scheme with eligibility rules, amount, deadline, seat count
FR-3	Admin can edit/close an existing scheme
FR-4	Admin can view all applications routed to them for final approval (status = `pending_admin_approval`)
FR-5	Admin can approve or reject an application, with an optional remark
FR-6	Admin can override a staff recommendation (with mandatory justification, logged)
FR-7	Admin can mark an approved application as `disbursed` (mock action)
FR-8	Admin can add/remove staff accounts and assign them
FR-9	Admin can view analytics dashboard with 4+ charts (see Section 4)
FR-10	Admin can view a searchable/filterable audit log of all actions

Non-Functional Requirements

ID	Requirement
NFR-1	All admin API routes protected by role middleware (`role === 'admin'`), checked server-side, not just hidden in UI
NFR-2	Every approve/reject/override action writes an entry to `AuditLogs`
NFR-3	Dashboard loads paginated data (don't fetch all applications at once)
NFR-4	Charts update from real DB data, not hardcoded values
NFR-5	Responsive layout — works on laptop screen sizes at minimum (demo-ready)

What You're Explicitly NOT Building

(so you don't scope-creep into a teammate's module)

- Student application form / upload UI
- Staff document verification screen
- Login/signup UI itself (assume Auth module gives you a working JWT + role — coordinate with whoever owns Auth on Day 1 morning)
- Notification delivery mechanism (you just call `createNotification()` — someone else builds the sender)

---

1. Module Breakdown

Module A — Scheme Management

Create, edit, close scholarship schemes.

Module B — Application Review & Approval

View pending-approval queue, approve/reject/override, trigger disbursement.

Module C — Staff Management

Add/remove/view staff accounts.

Module D — Analytics Dashboard

Charts and summary stats.

Module E — Audit Log Viewer

Searchable table of all system actions.

---

2. Day-by-Day Task List

DAY 1 — Morning (Setup + Module A)

- [ ] Sync with backend/Auth teammate: confirm JWT payload shape (`{ userId, role }`) and how role middleware will be shared across modules
- [ ] Confirm/finalize `Scholarship` schema fields with team (don't build in isolation — Student module reads this same table)
- [ ] Build API: `POST /api/scholarships` (create scheme)
- [ ] Build API: `PUT /api/scholarships/:id` (edit scheme)
- [ ] Build API: `PATCH /api/scholarships/:id/close`
- [ ] Build API: `GET /api/scholarships` (list, for admin's own management view)
- [ ] Frontend: "Create Scheme" form (title, eligibility rules, amount, seats, deadline, `requiresAdminApproval` toggle)
- [ ] Frontend: Scheme list table with Edit/Close actions

DAY 1 — Afternoon (Module B core)

- [ ] Build API: `GET /api/applications?status=pending_admin_approval` (paginated)
- [ ] Build API: `PATCH /api/applications/:id/approve`
- [ ] Build API: `PATCH /api/applications/:id/reject`
- [ ] Build API: `PATCH /api/applications/:id/override` (requires `justification` field, writes to audit log)
- [ ] Confirm with backend teammate how applications land in this queue (staff module must set status correctly — align on the state machine from the shared spec)
- [ ] Frontend: Approval queue table (student name, scheme, staff remarks, docs summary link)
- [ ] Frontend: Approve/Reject modal with remark field

DAY 1 — End of day checkpoint

- [ ] Demo Module A + B (even with dummy data) to your team lead — catch integration mismatches early, not on Day 2 night

DAY 2 — Morning (Module B disbursement + Module C)

- [ ] Build API: `PATCH /api/applications/:id/disburse`
- [ ] Frontend: "Mark as Disbursed" button on approved applications
- [ ] Build API: `POST /api/staff` (add staff), `DELETE /api/staff/:id`

- [ ] Build API: `GET /api/staff` (list with basic stats: applications assigned/completed)
- [ ] Frontend: Staff management table + "Add Staff" form

DAY 2 — Midday (Module D — Analytics)

- [ ] Build API: `GET /api/analytics/summary` — returns aggregated counts (total applications, by status, by scheme)
- [ ] Build API: `GET /api/analytics/trends` — applications over time (group by date)
- [ ] Frontend: Integrate Recharts/Chart.js — build these 4 charts minimum:
- [ ] Status breakdown (pie/donut)
- [ ] Applications over time (line)
- [ ] Approval rate by scheme (bar)
- [ ] Fund utilization per scheme (progress bars)

DAY 2 — Afternoon (Module E + Polish)

- [ ] Build API: `GET /api/audit-logs?actor=&action=&dateFrom=&dateTo=`
- [ ] Frontend: Audit log table with filters
- [ ] Full integration test: create scheme → (coordinate with student/staff teammates for a live application) → approve → disburse → check it reflects in analytics and audit log
- [ ] UI polish: loading states, empty states, error toasts
- [ ] Responsive check + remove console logs/dead code

DAY 2 — Final hours

- [ ] Deploy your module (or confirm it's merged into the combined deployed app)
- [ ] Prepare your 2–3 minute segment of the demo (show scheme creation → approval → analytics updating live — this is your most visually impressive sequence)
- [ ] Rehearse answer: *"How does this connect to what Staff/Student teammates built?"*

---

3. API Contract You Need From Teammates (confirm Day 1 AM)

You need this from...	What
Auth owner	JWT structure, role middleware function you can import/reuse
Staff module owner	Confirmation that `status` changes to `pending_admin_approval` correctly when staff verifies, and `assignedStaffId` is set
Student module owner	Confirmation of `Scholarship` schema field names match what their application form submits against

Do this handshake Day 1 first thing — mismatched field names between modules is the #1 hackathon integration failure.

---

4. Your API Endpoints — Full List (for your own reference/README)

```
POST   /api/scholarships
PUT    /api/scholarships/:id
PATCH  /api/scholarships/:id/close
GET    /api/scholarships

GET    /api/applications?status=pending_admin_approval
PATCH  /api/applications/:id/approve
PATCH  /api/applications/:id/reject
PATCH  /api/applications/:id/override
PATCH  /api/applications/:id/disburse

POST   /api/staff
DELETE /api/staff/:id
GET    /api/staff

GET    /api/analytics/summary
GET    /api/analytics/trends

GET    /api/audit-logs
```

---

5. If Reassigned: Equivalent Module Structure

If your team swaps you to **Staff** instead:

- Module A → Review Queue (view assigned applications)
- Module B → Document Verification (mark valid/invalid per document, with remarks)
- Module C → Eligibility Auto-Check display + manual override
- Module D → Recommend to Admin (submit decision)
- Skip analytics/audit-log ownership — usually stays with Admin module owner

If reassigned to **Student**:

- Module A → Browse/Search Scholarships
- Module B → Application Form + Document Upload
- Module C → Status Tracker (visual stepper: Submitted → Verified → Approved → Disbursed)
- Module D → Notifications inbox (read/unread)

- Skip staff/admin-only APIs entirely — you're a pure consumer of the `Applications` and `Scholarships` tables

The Day 1/Day 2 time split (setup+core → integration+polish) applies the same way regardless of which dashboard you land on.